Anil Neerukonda Institute of Technology & Sciences (Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email: principal@anits.edu.in

Department of CSE(AI&ML)

Course Code	Title of the course	Category	L	T	P	Total Instruction Hours	TW + SL	Total Hours	Sessional Marks	End Exam Marks	Total Marks	Credits
23CM9201	Java Programming Practices	PC	1	0	3	30	45	75	60	40	100	2.5

Prerequisites: Basic knowledge of computer fundamentals. Knowledge of some programming languages (such as C, C++)

Course Objectives:

- To Understand Object Oriented Programming Concepts and Apply Them in Problem Solving.
- To Learn the Basics of Java Console and GUI Based Programming.

Course Outcomes:

At the end of the course the student will be able to:

CO	Course Outcome Statement
CO-1	Develop classes for real time applications and establish the connectivity among the classes using inheritance and usage of strings.
CO-2	Modularize the application using packages, Interfaces and use of file concepts.
CO-3	Apply threads on classes to achieve parallelism through synchronization and develop test cases by including the runtime errors using exceptions handling mechanism.
CO-4	Solve the complex problems using collection framework.

Mapping of course outcomes with program outcomes:

COs	Program Outcomes (POs)												
Mapping	Domain Specific POs					Domain Independent POs						PSO	
CO	1	2	3	4	5	6	7	8	9	10	11	1	2
CO 1	1	2	2	-	-	-	-	-	-	-	-	2	-
CO 2	-	3	2	-	2	-	-	-	-	-	-	2	-
CO 3	-	3	2	2	-	-	-	-	-	-	-	2	-
CO 4	-	2	2	3	-	-	-	-	-	-	-	2	-
Correlation levels 1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High)													

Week-wise Theory and Lab Schedule:

Week – 1 & 2

Fundamentals of Object-Oriented Programming: Object oriented paradigm, Object and Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

Sample Program:

1. Bank Account Management System CO1

Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

Week – 3

Java Programming: Java Buzzwords, Data types, Variables, Operators, Control structures, Arrays, Console input and output.

Sample Program:

2. Java Basics CO1

1. Write a Java program to check whether the given number is prime palindrome or not.

2. Write a Java program which reads marks of five subjects through command line and print the total and average.

Week – 4 & 5

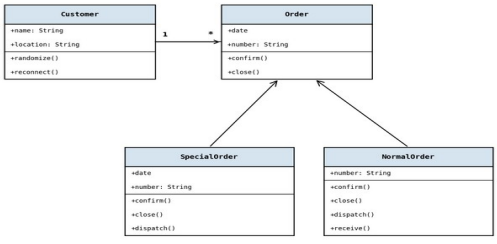
Introduction to Classes, objects, constructors, methods, parameter passing, overloading constructors and methods.

Sample Program:

3, 4, 5. Classes, Objects and Constructor Overloading CO1

3. Classes and Objects: Design a 'farm animals' Java application with details like where they stay, what they eat, the sound they make by using classes and objects.

4. Write a Java Program to implement the following class diagram:



5. Constructor Overloading: An organization is maintaining the data of employee according to cadre of employee with following parameters: name, id, designation, salary, promotion status. Apply the constructor overloading to implement it.

Week – 6

Static fields and methods, this reference, final keyword, String handling.

Sample Program:

6. Strings CO1

Write a program to find the longest Substring without Repeating Characters.

Input: abcabcbb Output: 3 string: abc

Input: pwwkew Output: 3 string: wke

Note: pwke is not a substring, it is a subsequence.

Week – 7

Inheritance – Types of inheritance, using super keyword, Member access rules, Preventing inheritance using final. Polymorphism – Dynamic binding, method overriding, abstract class and methods.

Sample Program:

7. Method Overriding CO1

Write a JAVA program to implement bank functionality and demonstrate dynamic polymorphism.

Note: Create classes namely Customer, Account, RBI (Base Class), SBI and derive Bank classes.

TestCase1: Enter the Bank name to find the rate of interest: RBI

RBI rate of interest is: 4%

TestCase2: Enter the Bank name to find the rate of interest: SBI

SBI rate of interest is: 7%

Week – 8

Interface – Defining an interface, implementing interfaces, accessing implementations through interface references, extending interfaces.

Sample Program:

8. Interfaces CO2

Different categories of employees are working in a software company like Regular Employees, Contract Employees and Vendors. Their pay is calculated differently.

Find the monthly salary details. If input is Regular Employee display the Regular employee salary details. If input is Contract based display the Contract salary details.

Week – 9

Packages – Defining, Creating and Accessing a Package, importing packages.

Sample Program:

9. Packages CO2

Define a package number and in that define Roman class and implement romanToInteger() and import the Roman class.

Input: 'III' Output: 3

Input: 'LVIII' Output: 58 (Explanation: L=50, V=5, III=3)

Week – 10

I/O – Reading and writing files.

Sample Program:

10. File Handling CO2

Write the below text in the file called sample.txt and then find the frequency count of the patterns 'pe' and 'pi'.

Peter Piper picked a peck of pickled peppers

A peck of pickled peppers Peter Piper picked

If Peter Piper picked a peck of pickled peppers?

Where's the peck of pickled peppers Peter Piper picked?

Expected Output:

'pe' – no of occurrences = 20 'pi' – no of occurrences = 12

Week – 11

Exception handling – Exception types, use of try and catch, throw, throws, finally, multiple catches, built-in exceptions, user defined exceptions.

Sample Program:

11. Exception Handling CO3

Input a mobile number and check whether the given number is a valid mobile number or not.

Rules:

- A valid mobile number is a combination of (0-9) digits of length exactly 10.
- If the Number Exceeds length of 10 raise: Invalid Mobile Number – ArrayIndexOutOfBoundsException.
- If the given Number is less than the length of 10 raise: Invalid Mobile Number – LengthNotSufficientException.
- If the given Number contains any character other than digit raise: Invalid Mobile Number – NumberFormatException.

Sample Input: 9885089465 → Valid number

98567890121 → ArrayIndexOutOfBoundsException

88664433 → LengthNotSufficientException

98abj@123 → NumberFormatException

Week – 12

Multithreading – Thread Priorities, synchronization, creating multiple threads.

Sample Program:

12. Multi-Threading CO3

Implement a Reservation system which allows the persons to book seats.

Define reserve method initially with 100 seats. Now create one or more person threads to book seats.

At any time it should allow only one person thread to access the reserve method.

Expected Output:

Person-1 entered. Available seats: 10 Requested seats: 5 → Seat Available. Reserve now :-) 5 seats reserved.

Person-2 entered. Available seats: 5 Requested seats: 2 → Seat Available. Reserve now :-) 2 seats reserved.

Person-3 entered. Available seats: 3 Requested seats: 4 → Requested seats not available :-(.

Week – 13

Collection Framework – The Collection Interface: List Interface, Set Interface, Queue Interface.

Sample Program:

13. Letter Combinations of a Phone Number CO4

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent.

A mapping of digit to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.

Example: Input: '23'

Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Week – 14

Collection Classes – ArrayList Class, LinkedList Class, HashSet Class, TreeSet Class, PriorityQueue Class.

Sample Program:

14. Arrays – Valid Parentheses CO4

Write a program to find the Valid Parentheses.

Given a string containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

Rules:

1. Open brackets must be closed by the same type of brackets.

2. Open brackets must be closed in the correct order.

Note that an empty string is also considered valid.

Input: () Output: valid

Input: ({ }) Output: Not valid

Text Book:

1. Herbert Schildt and Danny Coward, "The Complete Reference", 13th Edition, Tata McGraw Hill. (Exp: 1-14)

Reference Book:

1. P.J. Deitel and H.M. Deitel, "Java for Programmers", Pearson Education (OR) P.J. Deitel and H.M. Deitel, "Java: How to Program", PHI.
2. E Balagurusamy, "Programming with JAVA - A Primer", 6th Edition, Tata McGraw Hill.
3. P. Radha Krishna, "Object Oriented Programming through Java", Universities Press.
4. Bruce Eckel, "Thinking in Java", Pearson Education.

PSO: CSE(AI&ML)

PROGRAM SPECIFIC OUTCOMES (PSOs):

PSO 1: Apply Artificial Intelligence and Machine Learning concepts for the development of intelligent systems and applications to multidisciplinary problems.

PSO 2: Develop and implement automated solutions that leverage computing techniques to address real-world and societal problems.

Mapping of Course Outcomes with Program Outcomes – Java Programming Practices (23CM9201):

Course outcome	PO Mapped	Level Mapped	Justification for Mapping
CO-1	PO1	1	The student is able to apply the knowledge of computer fundamentals and OOP concepts in developing Java class hierarchies (Competencies:1.2.1). The student is able to apply computer science principles to implement inheritance and polymorphism in Java (Competencies:1.7.1).
	PO2	2	The student is able to analyze real-world problems and model them using OOP class structures (Competencies:2.5.1). The student is able to break down a system into interconnected classes and objects (Competencies:2.6.1).
	PO3	2	The student is able to design class hierarchies using inheritance and polymorphism for real-time applications (Competencies:3.6.2). The student is able to implement Java programs using constructors, method overloading, and string handling

			(Competencies:3.8.1).
	PSO1	2	The student is able to apply OOP principles for developing intelligent Java-based systems.
CO-2	**PO2**	3	The student is able to analyze modularization problems and identify appropriate package and interface structures (Competencies:2.5.1). The student is able to design modular systems with clear interface contracts (Competencies:2.6.1).
	PO3	2	The student is able to design modular Java applications using packages and interfaces (Competencies:3.6.2). The student is able to implement file-based data storage and retrieval in Java (Competencies:3.8.2).
	PO5	2	The student is able to identify and use modern Java tools and frameworks for building modular and maintainable applications (Competencies:5.4.1).
	PSO1	2	The student is able to develop modular and maintainable Java solutions for real-world problems.
CO-3	**PO2**	3	The student is able to analyze concurrent programming problems and identify synchronization strategies (Competencies:2.5.1). The student is able to break down complex multi-threaded systems into manageable thread structures (Competencies:2.6.1).
	PO3	2	The student is able to design exception handling frameworks and multi-threaded applications (Competencies:3.6.2). The student is able to implement synchronized thread communication mechanisms (Competencies:3.8.1).
	PO4	2	The student is able to analyze thread safety issues and optimize synchronization strategies (Competencies:4.6.1). The student is able to identify and resolve runtime errors using exception handling (Competencies:4.6.2).
	PSO1	2	The student is able to build reliable and concurrent Java applications using multithreading and exception management.
CO-4	**PO2**	2	The student is able to analyze complex data problems and identify appropriate collection structures (Competencies:2.5.2).
	PO3	2	The student is able to design solutions using appropriate collection classes such as ArrayList, HashSet, and TreeSet (Competencies:3.6.1). The student is able to implement algorithms using Java collection framework (Competencies:3.8.1).
	PO4	3	The student is able to analyze data structure performance using collections and optimize access patterns (Competencies:4.6.1). The student is able to interpret and validate results produced by collection-based algorithms (Competencies:4.6.2).
	PSO1	2	The student is able to design efficient data management solutions for intelligent system components.

SDG Mapping:

CO	SDG	SDG Title	Brief Linkage Explanation
CO1	SDG 4	Quality Education	Covers OOP principles, class design, inheritance and polymorphism, enabling students to acquire practical Java programming skills supporting quality technical education and lifelong learning.
CO2	SDG 9	Industry, Innovation and Infrastructure	Involves modular design using packages, interfaces, and file I/O, supporting scalable and maintainable software infrastructure, promoting innovation and modern industrial software practices.
CO3	SDG 8	Decent Work and Economic Growth	Focuses on exception handling and multithreading for robust concurrent applications, improving software reliability and system performance, enhancing employability through industry-relevant Java skills.
CO4	SDG 11	Sustainable Cities and Communities	Uses Java collection framework to enable efficient data management and algorithmic problem solving, supporting the development of scalable and sustainable software systems.

Credit-hour justification:

Component	Activity	Hours	Total
L	Lecture hours	15 x 1 = 15	15
T	Tutorial hours	-	-
P	Practical hours	15 x 3 = 45	45
	Sub total		**60**
TW	Assignment 1	-	-
	Assignment 2	-	-
	Pedagogy	-	-
	Group Discussion / Seminar	-	-
	Case Study	-	-
SL	Self-Learning (video lectures / online resources MOOCs / research papers) – Virtual Labs	15 x 1 = 15	15
	Assessment / Reflection / Viva / Quiz	-	-
	Sub total		**15**
	Grand total		**75**